

Software Project Management – IS212

User Stories

Announcements

Announcements – Slack

- Reminder that Slack is our primary communication platform
 - <https://is212-2026.slack.com/>
- You should be able to log in using Google SSO with your SMU account
- **Email me if not** (especially if you joined during BOSS 2/2A)
- **Please do:**
 - Add your section to your name
 - Upload a photo
 - Take part in #interesting-links and #help-is212



Announcements – Week 3 Quiz

- Our first quiz will take place at the **start of your Week 3 class**
 - *Starts promptly at the beginning of class!*
 - *No extra time if you arrive to class late*
- Need to be physically present (no make-ups; see Week 1 slides)
- Requires Respondus LockDown Browser
 - *Test your installation using the “dummy quiz” on eLearn*
- Short (~10 mins); covers the **lessons of Weeks 1–2**
 - *MCQ-style questions (no written)*

Announcements – Project Customer Briefing



IS212 (AY 2026/27 T1) – Customer Briefing

**** Please Read This First ****

For the IS212 project, you will build the first release of a system using the scrum process.

In this document, you will find a briefing from the (mock) customer describing their desired system.

Please note that -

This customer briefing is (initially) deliberately ambiguous. **Why?** No real-world customer knows *precisely* what they want on day 1, and there may be edge cases or conflicting expectations that have not yet been considered.

You can disambiguate the briefing by 'meeting' the customer (role played by your section's instructor) during the scheduled Q&A sessions over the next few weeks. Questions through other channels will not be answered.

Each section has a different 'customer'. Please only consider clarifications provided by your section's customer, as clarifications in other sections may not be consistent.

The customer briefing is quite wide-ranging. Don't panic! You will NOT build the entire system. The customer **will eventually clarify the core functionalities to cover in your first release** (i.e., your Week 12 submission). Wait for these to be identified in Week 4 before you start coding, and work on creating epics/user stories for your product backlog in the meantime.

Available in eLearn now

Start organising yourself into groups

Project Instructions – Wk3 Fri 6pm

What should **you do & **not do**
until Week 4...?**

Recap

Week 1 Recap – Software Development Processes

Traditional

- Waterfall
- RUP

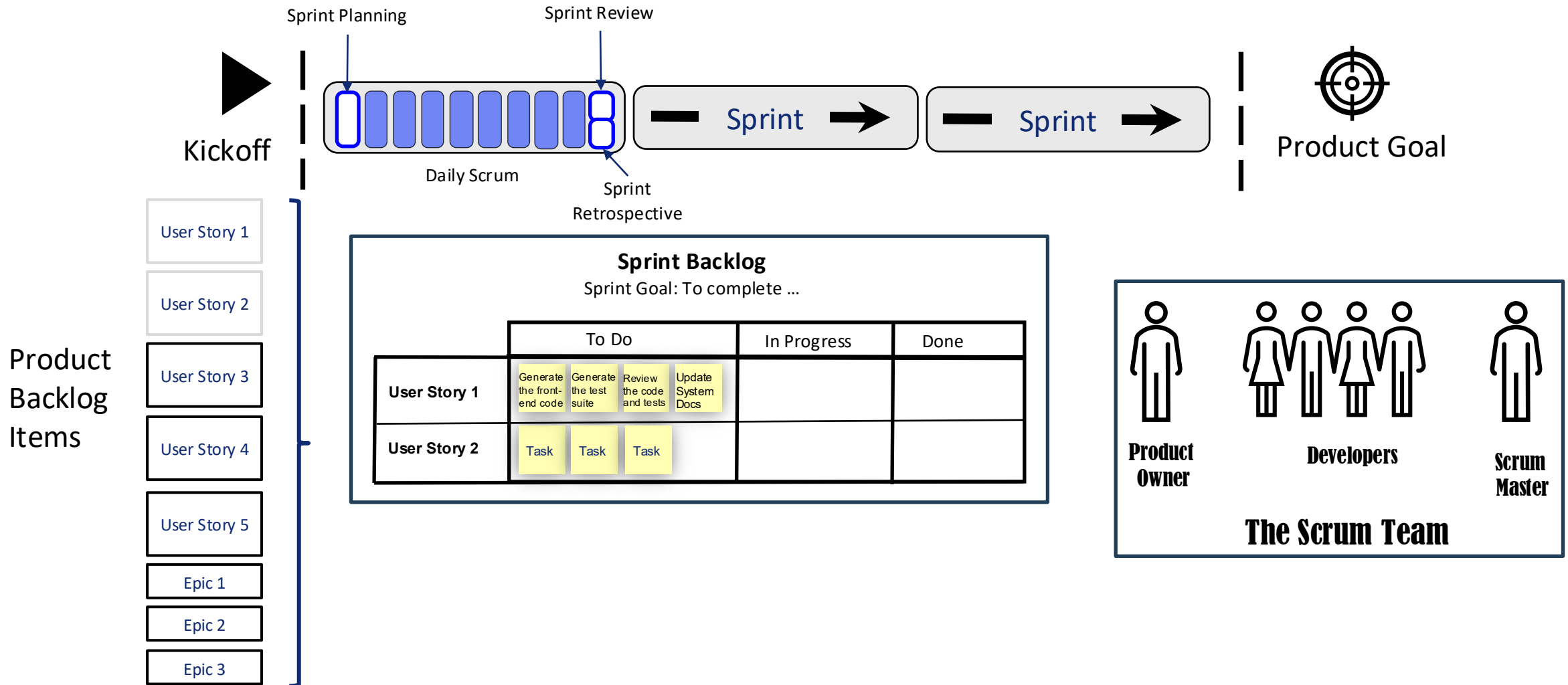
Agile

- Extreme Programming
- Kanban
- Scrum

Week 1 Recap – Software Development Processes

- **Waterfall** isn't dead: name a project where **you'd choose it**, and why?
- Which software development processes are **iterative**? 
- **XP**'s idea was to **take good practices to a "logical extreme"**. What does that actually mean?
- In **Scrum**, why is the **sprint goal** frozen for the whole sprint, when agile is supposed to **welcome change**?
- **Scrum** vs **Kanban**: similar-looking **boards** – what's the difference?
- What is the **impact of AI-assisted coding** on software development processes?

Week 1 Recap – Scrum



User Stories

User Stories

Objectives

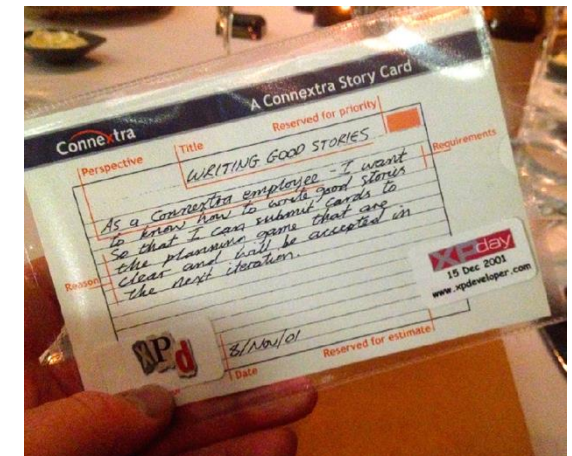
On completing this class, you will be able to:

- Explain the rationale for writing **user stories** and **epics**
- Create user story cards to **describe the features** of a system
- Evaluate the quality of a user story card against the **“INVEST” criteria**
- Judge when to use AI to **draft, critique, and split** stories – and when not to

Outline

- User Stories / User Story Cards
- INVEST Criteria
- Epics, Story Maps, and Splitting
- User Stories in Scrum

Product Backlog





**Clearly identifies
the user,
their desire, and
their intention**



<https://www.youtube.com/watch?v=VKFKypbvoUI>

What is a User Story?

- A **user story** is a brief, simple description of a **feature** or functionality from the **perspective of an end user**
- It typically follows this format:

As a *<role>*, I want to *<perform an action>* so that I can *<achieve a goal or outcome>*


Who


What


Why

Examples – Learning Management System (à la eLearn)

User Story #1

*As a **student**, I want to **submit my group project files** so that **my team gets credit for the work we finished before the deadline**.*

User Story #2

*As an **instructor**, I want to **download my section's group project files** so that **I can work through them all in one sitting**.*

What are the **benefits** of describing features as user stories?

*user stories use **non-technical** language to provide context for the whole team*

*user stories put **end users** at the centre of the conversation*

What are the **benefits** of describing features as user stories?

*they convey the **value** of the feature: helpful for prioritisation*

*with the end goal defined, teams can decide how to **collaborate** together to achieve it*

*in short: they facilitate a **conversation*** 

Can't AI Just Write These for You?

- Paste the customer briefing into an LLM and you will have **twenty user stories in ten seconds**
- They will look right: correct format, plausible roles, clear value
- But which of them...
 - describe something the customer never actually asked for?
 - assumed the answer to a question nobody has yet asked?
 - are the ones worth building first?

***Generating stories is now cheap.
Judging them is not.***

Card, Conversation, Confirmation

- Ron Jeffries' **three Cs**: a user story is not the text on the card – it is three things
 - **Card** – a short written placeholder for the requirement
 - **Conversation** – the discussion that fills in the detail (the whole point!)
 - **Confirmation** – the acceptance criteria (introduced shortly) that record what 'done' means
- AI writes the **Card** in seconds
- AI **cannot** replace the **Conversation** – it has never met your customer
- So the **Confirmation** is where your work now goes 

As a <role>

“As a **customer**, I want to pay by credit card online, so that I can complete my purchase conveniently.”



- Specify an abstract **role**, rather than a specific person or job title
- Helps with **generalisation**, e.g. thinking about the feature for *any* customer on the system
- Encourages the team to **converse** about a feature from the perspective of the users who derive value from it

What about features involving
two or more roles?

I want to <perform an action>

“As a customer, I want to **pay by credit card online**, so that I can complete my purchase conveniently.”



- Specifies **what** the user wants to do
- For example:
 - Want to **do something**, e.g. see list of credit card transactions
 - Want to **get something**, e.g. get alert for each credit card transaction
 - Format: {Active verb} {Object}, e.g.:
 - {see list} of {credit card transactions}
 - {get alert} for each {credit card transaction}
 - Object qualifiers, e.g.:
 - can get alert for each credit card transaction of \$100 or above

Is this enough to know what needs to be built?

So that I can <achieve a goal or outcome>

“As a customer, I want to pay by credit card online, so that I can **complete my purchase conveniently**.”



- Important context – the reason **why** we're building this feature
- (Hopefully) have **better conversations** among the scrum team and customer to converge on a better solution
- For example – what does “conveniently” mean here?
 - Different credit card providers?
 - Purchase without the need for additional OTPs (faster)?
 - Card details saved?

User Stories – Rigid Format, Flexible Phrasing

- As a traveller, I **want** to search for flights by dates so that I can find the most affordable options for my trip
- As a student, I **want** to access online learning materials offline so that I can study even when I do not have an internet connection
- As a homeowner, I **can** schedule automated lighting control so that I enhance security whilst I am away
- As a driver, I **can** receive real-time traffic updates so that I avoid congestion
- As a job applicant, I **need** to upload my resume as an attachment so that I can apply without retyping my whole history into a form
- As a social media user, I **need** to report abusive accounts so that I can maintain a safe online experience

User Story Card

User Story Card

Title <i>Good short title</i>	Priority <i>Order of implementation</i>	Estimate See Week 3!
User Story <i>Simple description (no details) of customer/user need (conversation starter)</i> <i>As a <role>, I want to <perform an action> so that I can <achieve a goal or outcome></i>		
Acceptance Criteria <i>List of conditions fulfilled on completion of implementation</i>  - Boundaries of the story - Use to check if the story ‘done’		

User Story Card – Example

Title <i>Accept Credit Card Payments</i>	Priority <i>(See Week 3...)</i>	Estimate <i>(See Week 3...)</i>
User Story <i>“As a customer, I want to pay by credit card online, so that I can complete my purchase conveniently.”</i>		
Acceptance Criteria Ensure that the customer: • Can input a valid card number, expiration date, CVV, and cardholder name • Can only pay with Visa, Mastercard, or American Express • Is requested to provide an OTP • Has their payment processed and their order confirmed • Is informed of payment success or failure within 2 seconds • Has a secure TLS connection		

Acceptance Criteria (ACs)

- **Acceptance criteria** define the conditions that must **ALL** be met for a user story to be considered **complete** and **acceptable**
 - *They should be clear, unambiguous, but not overly specific*
 - *They should be **measurable** / **testable***
- They define the '**boundaries**' of the user story
 - *What does 'done' look like from the user's perspective?*
 - *What is included within the **scope** of the story? What is **NOT**?*
- Used to **build consensus** within the scrum team
 - *Common understanding on what needs to be built*
 - *In turn, this helps with **better planning and estimation** (Week 3)*
- ACs can serve as the basis for **test case design** (Week 4)
- ACs can also serve as the **spec that an AI agent builds against** (shortly!)

What's meant by measurable / testable? Why important?

User Story

Acceptance Criteria

Acceptance Criteria – Given-When-Then

- At times, it can be helpful to elaborate on an acceptance criterion using the **given-when-then** format (aka **scenario format**)
 - Describes a condition (**given**) under which a triggering event occurs (**when**) resulting in an outcome (**then**)

Checklist-based AC

- Is immediately informed of payment success or failure

Scenario-based ACs (form of Given-When-Then)

- **Given** a customer has entered valid credit card details
When they confirm payment
Then they should receive a payment confirmation message
- **Given** a customer has entered incomplete/invalid card details
When they confirm payment
Then a message should advise them to check their card details and payment should not be processed
- ...

Checklist- or Scenario-based ACs?

- **Checklist-based ACs** are **concise** and faster to write (**more agile**)
 - Useful when the requirements are simple and do not involve complex user interactions
 - Less can be more! Leaves more detail open to scrum-team negotiation
- **Scenario-based ACs** provide **more context** on specific user interactions
 - Useful when the requirements are complex or involve multiple user journeys
 - Or when it's important to understand system behaviour across different conditions
- **Which one to use?** Either or a mix of both, **as needed**

Your Story Card is the Spec

- When an AI agent writes the code, the story and its acceptance criteria are **what you hand it**
- A growing practice is specification-driven development ("**spec-driven development**"), where AI agents implement from structured specifications
 - Write the spec, **let the agent build against it**
 - tooling such as GitHub Spec Kit and AWS Kiro – a disciplined answer to 'vibe coding'
 - some teams write **scenario-based ACs** in **EARS** notation, which constraints natural language by imposing a consistent clause order and using a limited set of bolded keywords (**WHILE**, **WHEN**, **IF**, **THEN**, **WHERE**, and **SHALL**)
 - e.g. '**WHEN** the customer confirms payment **THE SYSTEM SHALL** report success or failure within 2 seconds'
- No acceptance criteria ⇒ no way to tell whether the agent built the **right** thing
- A spec is **not** a guarantee: agents still ignore them, so you still review

From User Story Card to Prototype

- User stories are supposed to **facilitate conversations**
- Conversations can be sharper if the story is **accompanied with a prototype**
 - Coding-up simple prototypes to build consensus with stakeholders is a long-standing practice of **eXtreme Programming (XP)**
- AI coding allows simple user story **prototypes to be built within minutes**
 - Get stakeholder feedback faster before committing time to engineering it properly
 - ***“Wait, that’s not what I meant!” ... “Oh, but what if the user wants to do X?”***

***Demo: let’s generate a prototype
for one of our user story cards***

INVEST Criteria

(i.e. characteristics of good stories)

Writing Good User Stories

Independent

Negotiable

Valuable

Estimable

Small

Testable

A good **user story card** satisfies six attributes.

Bill Wake, *'INVEST in Good Stories, and SMART Tasks'* (2003), coined the **INVEST** acronym for ease of recall.

Writing Good User Stories

Independent

Independent

Valuable

Interdependent – there's nothing to alert on until you've selected something

Small

Testable

Ensure User Stories are as independent as possible.

The effort required for each User Story can then be estimated and selected for inclusion in a Sprint **without regard for any other User Stories**.

1. As a patient, I can see all *my bills* sorted on date and institution...
2. As a billing executive, I can *create bills* showing different dates for admitting, discharging, billing, invoicing, reminding and paying...
3. As a diner, I can select which restaurant offers I want to follow....
4. As a diner, I can receive alerts for the restaurant offers I have selected...

How do we build (1) without also building (2) ?

Interdependent User Stories make estimation and planning more complicated.

Writing Good User Stories

Independent

A User Story is **negotiable**: it describes the desired outcome, not a fixed solution. It is a **placeholder for conversation**, and its details may be refined as the team learns more.

Negotiable

Even during a Sprint, details may be clarified or renegotiated with the Product Owner, provided the **Sprint Goal** is not endangered.

Valuable

Typical questions include:

Estimable

- Is the story still valuable?
- Is it too large or too small?
- Are the acceptance criteria sufficient?

Small

- Do the acceptance criteria **over-specify** how the solution must be implemented?

Testable

Can you think of ACs that violate the 'negotiable' property?

Writing Good User Stories

Independent

Negotiable

Valuable

Estimable

Small

Testable

“Valuable” is from the viewpoint of the role for which the User Story is written. The User Story can deliver meaningful value in the form of increased efficiency, improved experience, cost savings, revenue generation, meeting a specific business requirement, etc.

An example,

As a subscriber, I can be alerted to my unpaid bills, so I can pay punctually and not be charged penalty

This User Story is valuable in that it can potentially save the subscriber penalty charges.

Why do we care about the value of user stories?

Writing Good User Stories

Independent

Negotiable

Valuable

Estimable

Small

Testable

User Stories should be clear enough so that developers can reasonably estimate the required effort.

When might User Stories be unclear:

- **Developers lack domain knowledge** – have User Story conversations to elaborate and specify to the level needed to make an estimate
- **Developers lack technical knowledge**, e.g.,

As a security officer, I want communications to be protected by TLS, so that sensitive information cannot be intercepted

Take a time-out to investigate TLS, but strictly within a timebound

- **User Story is too big**, e.g.,

As a student, I want to communicate with the instructor to schedule a consult, to clarify doubts

Break it into bite-sizes

As a student, I want to email the instructor to schedule a consult...

As a student, I want to Telegram the instructor to schedule a consult...

As a student, I want to Slack DM the instructor to schedule a consult...

Is it easier to estimate small work items or large work items?

Writing Good User Stories

Independent

Negotiable

Valuable

Estimable

Small

Testable

Think Goldilocks, not too small, not too big, just the right size for planning:

- If it is too big, it cannot fit within a Sprint, e.g.,
As a patient, I want to see all information the hospital has of me...

Break it into bite-sizes

As a patient, I want to see all my bills...

As a patient, I want to see all my medical records...

- If it is too small, it might cause inefficiency or lack functional value, e.g.,
As a patient, I want to see total subsidies...

As a patient, I want to see all subsidy line items...

As a patient, I want to see all charges up to two decimal places...

Club them up to modular size

Which other INVEST properties is this connected to?

Writing Good User Stories

Independent

“If the story cannot be tested, how can the developers know when they have finished coding?” – Mike Cohn, *User Stories Applied*

Negotiable

There must be a **clear set of acceptance criteria** for what success looks like and the means to test the criteria:

Valuable

- The screen must not take too long to appear – this is **untestable**; how long is long, one second, two seconds, etc.
- The screen must appear within 2 seconds – this is testable

Estimable

Small

Testable

INVEST When an Agent Writes the Code

- The six attributes still hold – but their **weights shift**:
 - **Testable** ↑ the AC is now the contract the agent builds against, and how you check its work
 - **Valuable** ↑ anyone can generate a story; deciding what **not** to build is the scarce skill
 - **Independent** ↑ agents touch many files at once – independent stories keep changes reviewable
 - **Small** ∪ re-anchored: small enough to **review**, not small enough to type
 - **Estimable** ? if writing code got faster but reviewing it didn't, what are we estimating? (Week 3)
 - **Negotiable** ⚠ over-specifying the AC to please the agent removes the team's room to negotiate

Epics & Splitting Stories

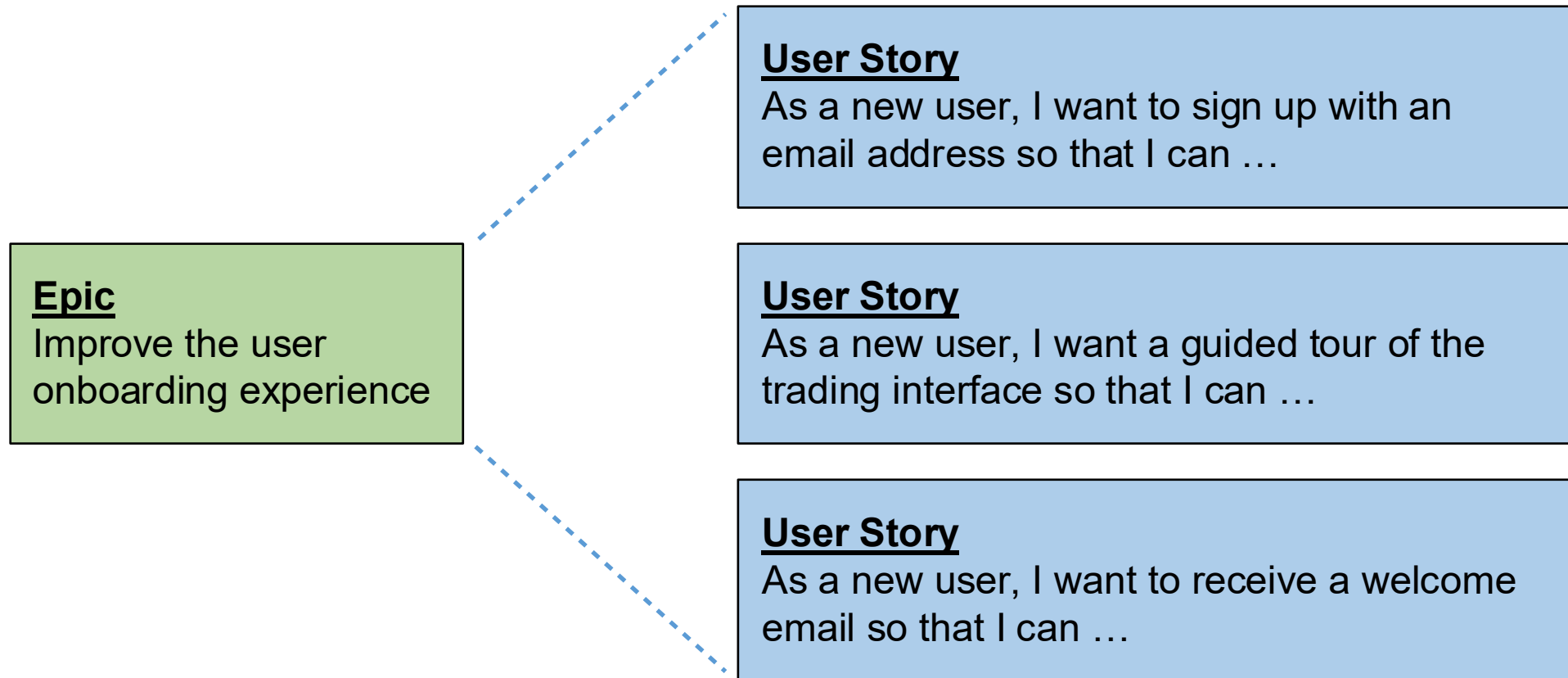
Epics

- Suppose you are implementing a new LMS (e.g. eLearn) for a customer
- The customer has indicated that they **want it to have a message board**, but that this is a **very low-priority** feature
- Writing INVEST-able user stories for a message board will require a lot of time. Think of the possible features:
 - Create post, delete post, edit post, notification, swear filter, ... 🤯
- **Is it worth the effort to create stories for all these features** when your team has many other more pressing features to prioritise?
- **No?** But shouldn't the **product backlog list all required functionality** ... ?

Epics

- A solution is to use “**epics**”, which are large stories that represent a **broad objective** or **significant piece of functionality**
- Use epics as a **placeholder** on the product backlog until they are ready to be replaced by (multiple) INVEST-able user stories
- Epics can also be used to help **organise your user stories**
 - i.e. associate each user story card with a particular overarching epic

Epics – Example



Right-sizing your user stories

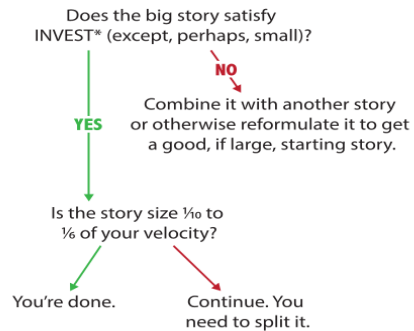
- Suppose your user story **violates** the “S” of INVEST
 - i.e. it’s “too large”
- **Splitting** it into two (or more) smaller, INVEST-able user stories gives you **more freedom to plan** when they will be implemented
- “**Splitting patterns**” provide some guidance on how to do this



HOW TO SPLIT A USER STORY

1

PREPARE THE INPUT STORY



* INVEST - Stories should be:
Independent
Negotiable
Valuable
Estimable
Small
Testable

WORKFLOW STEPS

DEFER PERFORMANCE

Could you split the story to just make it work first and then enhance it to satisfy the non-functional requirement?

Does the story get much of its complexity from satisfying non-functional requirements like performance?

SIMPLE/COMPLEX

Could you split the story to do that simple core first and enhance it with later stories?

Does the story have a simple core that provides most of the value and/or learning?

MAJOR EFFORT

Could you group the later stories and defer the decision about which story comes first?

Does the story describe a workflow?

Can you split the story so you do the beginning and end of the workflow first and enhance with stories from the middle of the workflow?

Can you take a thin slice through the workflow first and enhance it with more stories later?

OPERATIONS

Can you split the operations into separate stories?

Does the story include multiple operations? (e.g. is it about "managing" or "configuring" something?)

BUSINESS RULE VARIATIONS

Can you split the story so you do a subset of the rules first and enhance with additional rules later?

Does the story have a variety of business rules? (e.g. is there a domain term in the story like "flexible dates" that suggests several variations?)

VARIATIONS IN DATA

Can you split the story to process one kind of data first and enhance with the other kinds later?

Does the story do the same thing to different kinds of data?

Does the story have a complex interface?

Is there a simple version you could do first?

INTERFACE VARIATIONS

Can you split the story to handle data from one interface first and enhance with the others later?

Does the story get the same kind of data via multiple interfaces?

When you apply the obvious split, is whichever story you do first the most difficult?

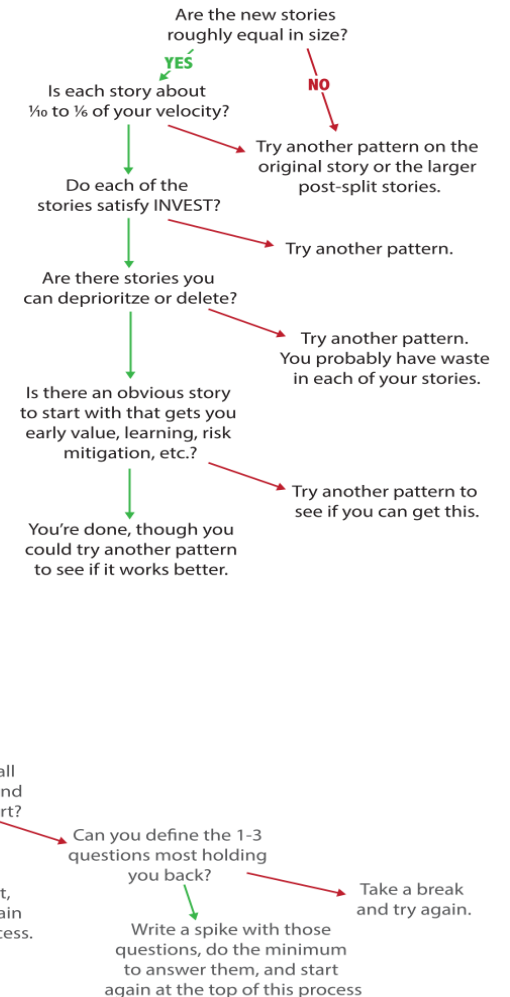
2 APPLY THE SPLITTING PATTERNS

start here

last resort

3

EVALUATE THE SPLIT



Splitting a User Story – Example

Title <i>Good short title</i>	Priority <i>Order of implementation</i>	Estimate <i>“Ideal effort” to implement</i>
Show Cinema Location & Movie Schedule	<i>(Release/Sprint Planning Use)</i>	<i>(Estimation Use)</i>
User Story <i>Simple description (no details) of customer/user want (conversation starter), e.g.,</i> As CinemaManager, I want to show the cinema’s address and movie schedule, so movie goers will visit Too big; “and” is the logical split		
Acceptance Criteria <i>List of conditions fulfilled on completion of implementation, e.g.,</i> • Single page showing cinema name, address, nearest MRT and map • List of movies: title, starring, synopsis, show times, ticket charges		

Splitting a User Story – Example

Title <i>Good short title</i> Show Cinema Location	Priority <i>Order of implementation</i> <i>(Release/Sprint Planning Use)</i>	Estimate <i>“Ideal effort” to implement</i> <i>(Estimation Use)</i>
User Story <i>Simple description (no details) of customer/user want (conversation starter), e.g.,</i> As CinemaManager, I want to show the cinema’s address, so movie goers know how to get here		
Acceptance Criteria <i>List of conditions fulfilled on completion of implementation, e.g.,</i> Single page showing cinema name, address, nearest MRT and map		

Has
WHO, WHAT , WHY

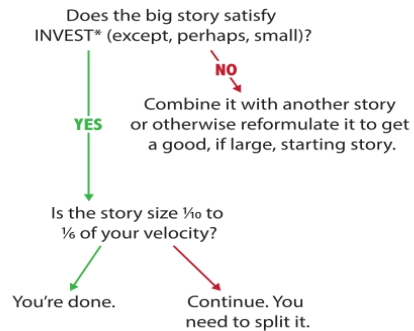
And
criteria for “done”

The movie schedule feature then becomes a different user story card

HOW TO SPLIT A USER STORY

1

PREPARE THE INPUT STORY



* INVEST - Stories should be:
Independent
Negotiable
Valuable
Estimable
Small
Testable

WORKFLOW STEPS

DEFER PERFORMANCE

Could you split the story to just make it work first and then enhance it to satisfy the non-functional requirement?

Does the story get much of its complexity from satisfying non-functional requirements like performance?

SIMPLE/COMPLEX

Could you split the story to do that simple core first and enhance it with later stories?

Does the story have a simple core that provides most of the value and/or learning?

MAJOR EFFORT

Could you group the later stories and defer the decision about which story comes first?

When you apply the obvious split, is whichever story you do first the most difficult?

2 APPLY THE SPLITTING PATTERNS

OPERATIONS

Can you split the operations into separate stories?

Does the story include multiple operations? (e.g. is it about "managing" or "configuring" something?)

BUSINESS RULE VARIATIONS

Can you split the story so you do a subset of the rules first and enhance with additional rules later?

Does the story have a variety of business rules? (e.g. is there a domain term in the story like "flexible dates" that suggests several variations?)

VARIATIONS IN DATA

Can you split the story to process one kind of data first and enhance with the other kinds later?

Does the story do the same thing to different kinds of data?

Does the story have a complex interface?

INTERFACE VARIATIONS

Can you split the story to handle data from one interface first and enhance with the others later?

Does the story get the same kind of data via multiple interfaces?

Is there a simple version you could do first?

BREAK OUT A SPIKE

Are you still baffled about how to split the story?

Can you find a small piece you understand well enough to start?

Write that story first, build it, and start again at the top of this process.

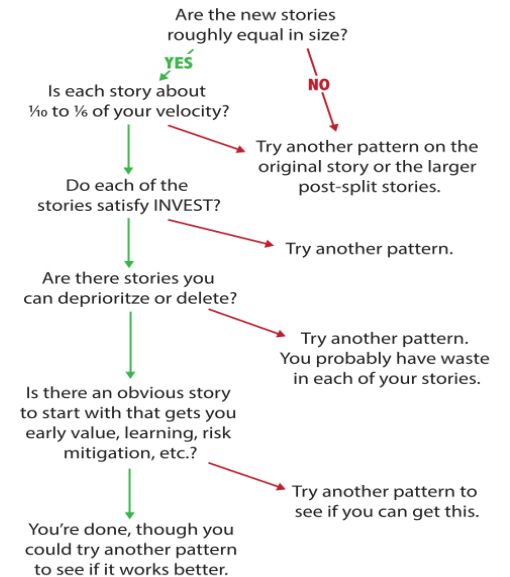
Can you define the 1-3 questions most holding you back?

Write a spike with those questions, do the minimum to answer them, and start again at the top of this process

Take a break and try again.

3

EVALUATE THE SPLIT



User Story

*As a **user**, I can **manage my account** so that ...*

—> “manage” implies CRUD. Apply the “Operations” splitting pattern

User Story

User Story

User Story

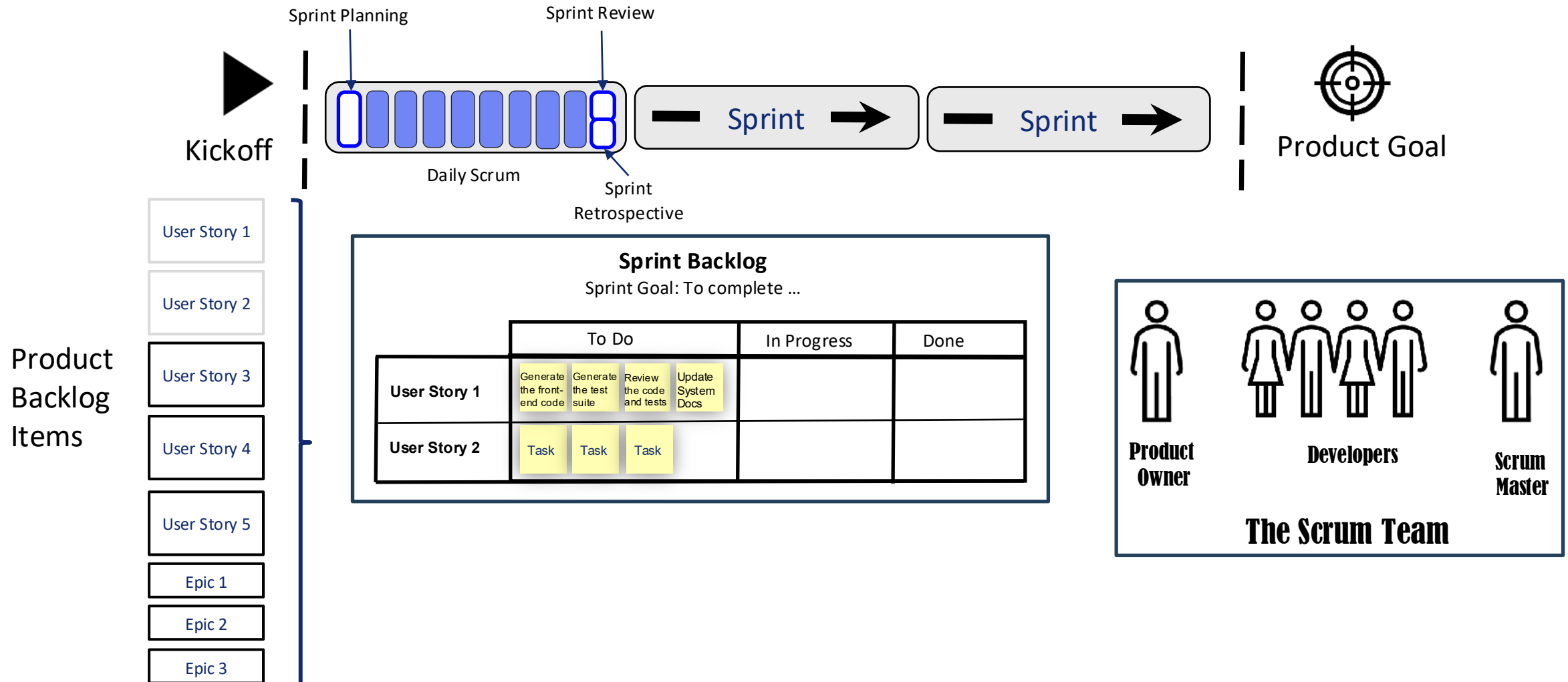
Using AI on the Backlog: Critic, not Author

- **Good** – AI as a **critic**:
 - audit a story against INVEST: 'which of these ACs is not testable, and why?'
 - hunt for ambiguity: 'list every term here that two people could read differently'
 - generate edge cases and error scenarios the team missed; propose splits using the patterns above
- **Poor** – AI as an **author**:
 - a bulk-generated backlog nobody has read – 200 plausible stories is not a backlog
 - scope the customer never asked for; false precision, e.g. an invented 'loads within 2 seconds'
- A possible modern workflow:

AI drafts - - - - - ➤ ***Team decides*** - - - - - ➤ ***Customer confirms***

User Stories in Scrum

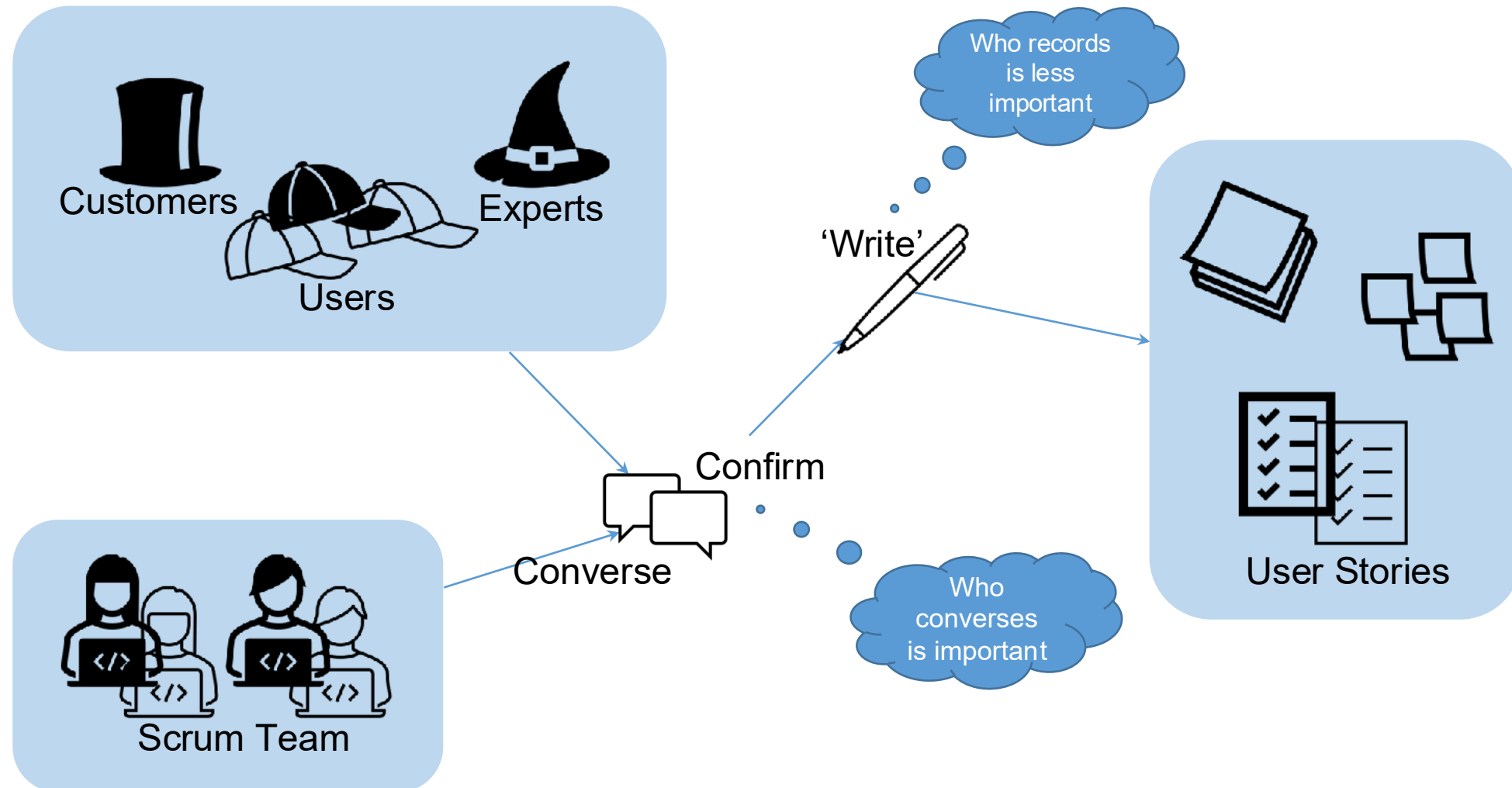
User Stories & Epics are Groomed in the Product Backlog



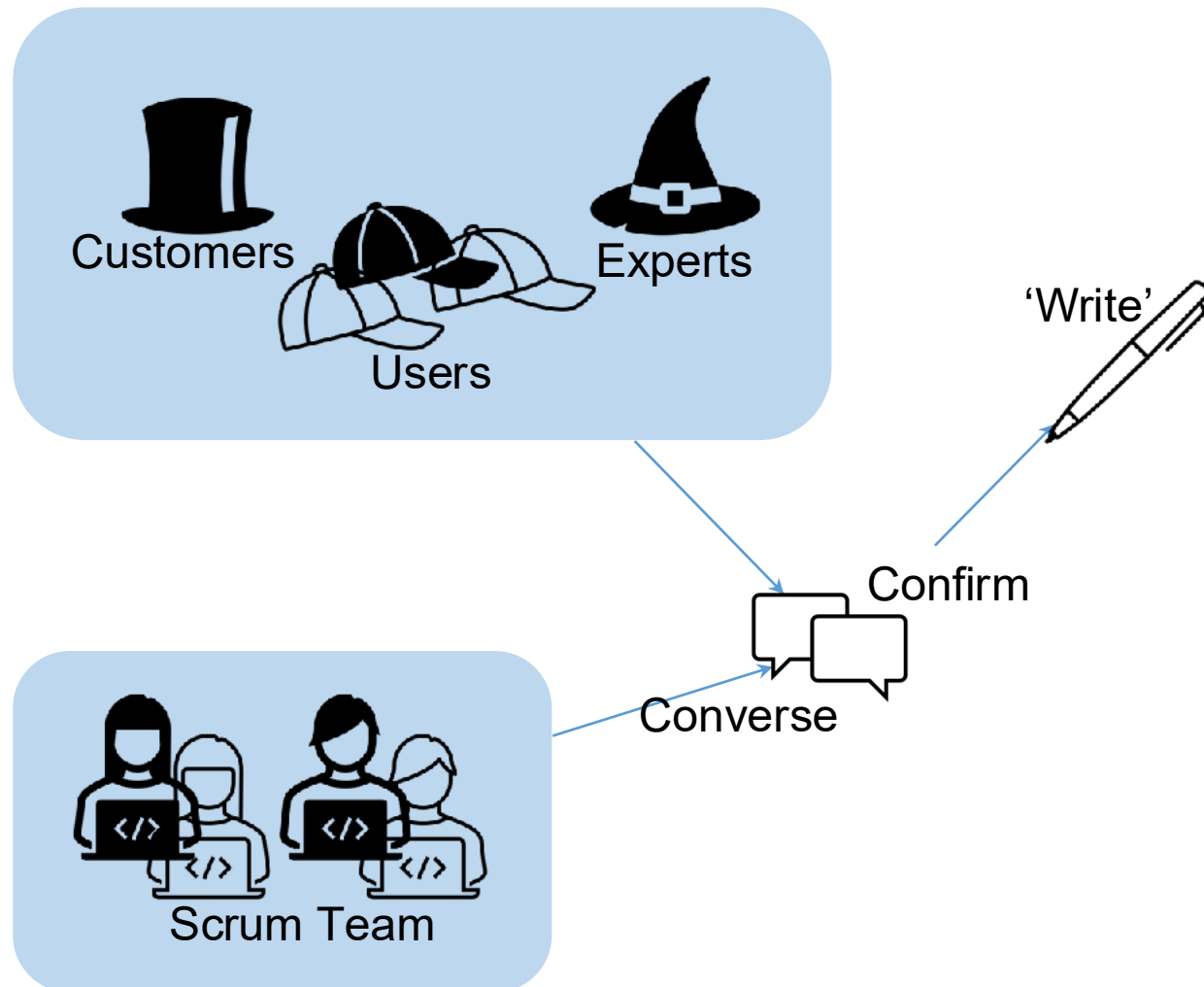
User Stories & Epics are Groomed in the Product Backlog

- The product backlog is a **living artefact** – it reflects the team's **current understanding** of what needs to be built
- User stories and epics can be updated, split, re-estimated, or re-prioritised in the product backlog **at any time**
- **Conversations** within the team and with the customer are key to this process
- In scrum, which team member has **ultimate responsibility** for the product backlog?

User Stories & Epics are Groomed in the Product Backlog



User Stories & Epics are Groomed in the Product Backlog



Mike Cohn...

'Software requirements is a communication problem...'

'When the business side dominates, it mandates functionality and dates with little concern that the developers can meet both objectives...'

'When the developers dominate...technical jargon replaces the language of the business...developers lose the opportunity to learn what is needed by listening...'

'If either side dominates these communications, the project loses...'

How 'Conversations' Go

Roles

- There is no one 'User'
- There are many 'Roles' each with their own needs
 - What are these roles?
 - Name the roles
 - Group them based on similar pattern of use
 - Rename intuitively as needed
 - Use these roles to enumerate user stories

User Stories

- User Stories are not all obvious or of the same size
- They can be apparent at **different moments**, be of different size and will **change with time**
- The aim is to capture as many as apparent with the thought that there are more
- As with traditional business analysis, **various techniques** are useful to trawl for user stories
 - **Questionnaire** – for a start with large user base
 - **Observation** – insights from visible/invisible observation and/or user demonstration
 - **User Interview** – structured, focused, non-leading interview of curated users
 - **Story-writing Workshop** – whole team lo-fidelity brainstorming, prototyping, role-playing, etc.

How Much to Write?

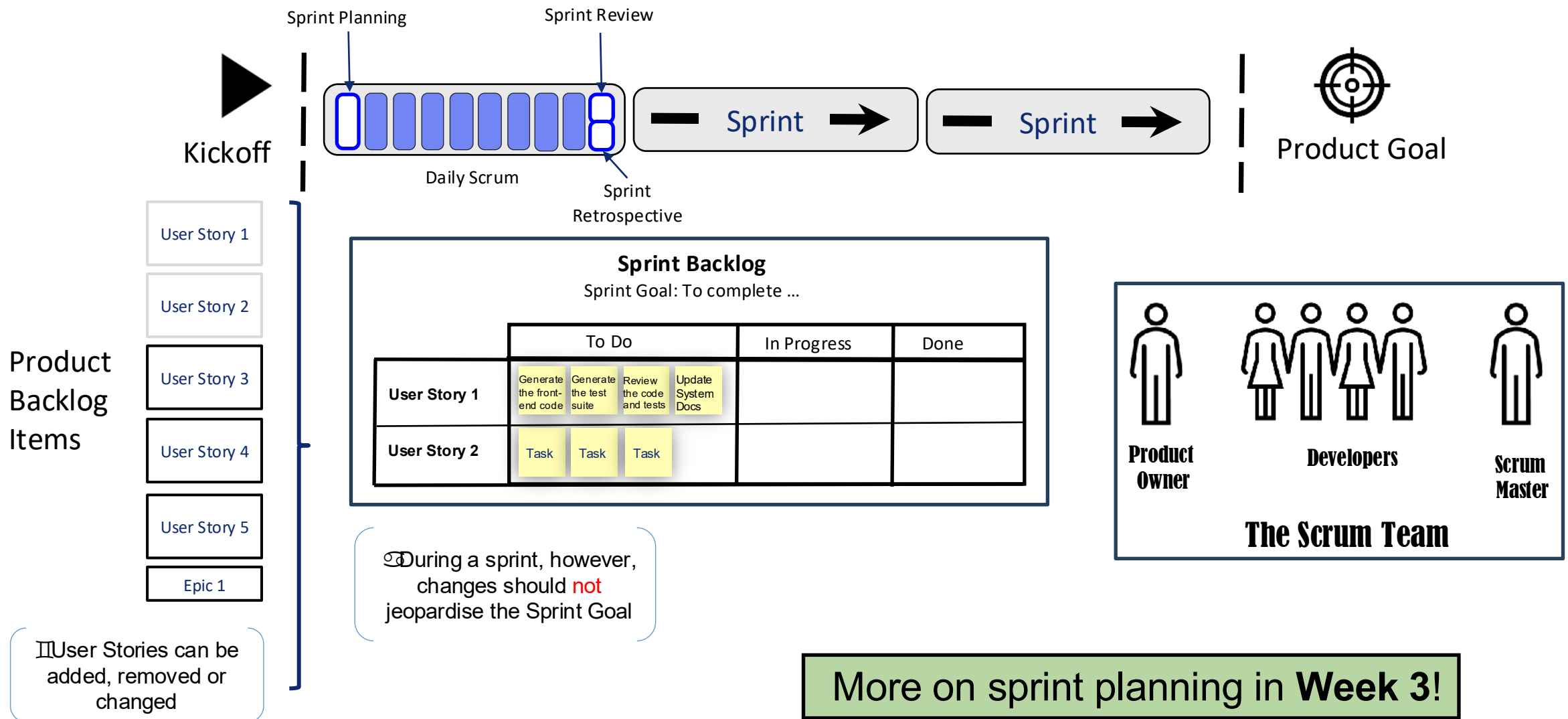
A good product backlog is...



How Much to Write?

- **Detailed appropriately** – the top priority items are more fine-grained and detailed than the lower priority items, since the former will be worked on sooner than the latter. For example, the top 10% of the backlog may be composed of very small, well-analysed items, and the other 90% much less so
- **Estimated** – the items for the current release need to have estimates, and furthermore, should be considered for re-estimation each Sprint as everyone learns and new information arises. **(Week 3)**
- **Emergent** – in response to learning and variability, the Product Backlog is regularly refined. Each Sprint, items may be added, removed, modified, split, and changed in priority. Thus, the Product Backlog is continuously updated by the Product Owner to reflect changes in the needs of the customer, new ideas or insights, moves by the competition, and technical hurdles that appear.
- **Prioritised** – the items at the top of the Product Backlog are prioritized or ordered in a 1-N order. In general, the highest-priority items should deliver the most bang for your buck: lots of bang (business value) for low buck (cost). Another motivation to increase the priority of an item is to tackle high risks early, before the risks attack you

What about the Sprint Backlog?



Who is Accountable for the Story?

- **Traceability:**

- user story → acceptance criteria → test case (Wk 4) → test class (Wk 6) → code

- When an agent wrote the code, that chain is your evidence of **what** you built and **why**
- In **Week 13** we will pick a feature of your project at random and ask your team to walk us through the chain
- 'The agent decided' is **not** an answer – your team is accountable for what it ships

Lab & Assignment

Lab Exercise – Product Backlog in Jira

- Access the **Week 2 Lab Exercise** in eLearn
- It will show you how to create a product backlog in **Jira**, a popular project management platform
- **This is for illustration only** – you can use any project management software you like for the course project:
 - *Well-managed Google Sheets are also fine!*



Week 2 Lab – Creating a Product Backlog in Jira

In this lab, we will be setting up a **product backlog** using Atlassian's popular project management software, **Jira**.

In particular, we are going to create a new Jira workspace and set up your first scrum project.

Setting Up a Jira Account and Scrum Project

In the first step, we are going to create a new Jira workspace and scrum project.

Please follow the steps below:

1. Go to: <https://www.atlassian.com/software/jira/pricing>
2. Select "Get it Now" under the Free plan.
3. Either log in to your existing Atlassian/Jira account, or create a new one. You can use Google SSO to connect it with your SMU email address.
4. Choose a URL for your workspace, e.g. "yourusername-is212.atlassian.net"
5. Wait a minute or two for everything to be set up.
6. Answer a few questions about yourself, then select "Scrum" as the template.
7. Choose a name for your scrum project, e.g. "IS212 Labs".

At this point, your new Jira scrum project should be ready to experiment with.

Individual Assignment #1 – User Stories

- Individual Assignment #1 is on eLearn
- In your teams, **generate** a pool of candidate user stories for the customer briefing
- **Individually**, choose three stories from that pool and refine them to **satisfy INVEST**
 - *i.e. “Divide and conquer!”*
- Worth 2.5% of the grade for IS212

IS212 (AY 2026-27, T1) – Week 2: User Story Assignment

Background

Last week, we received a (wordy!) briefing from the ‘customer’ about the system they want your group to implement for the IS212 project.

Before jumping into coding, it is important to build some consensus on *what exactly needs to be built*. To achieve this, we will be creating some **user story cards** (as introduced in this week’s class) to describe the features of the system desired by the customer.

These user story cards will be based on our **current understanding** of the system. In later weeks (i.e. after this assignment), you will be able to refine the user story cards further as the customer starts answering your questions and providing more details/context¹.

Tasks

The goal of this assignment is to seed your team’s product backlog with a set of high-quality user story cards, and to give you practice reviewing and refining stories against the INVEST criteria.

Due 23:59 four days after class
(1 sec late = 50% pen.)

Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet 🖥️
- <http://smu.sg/is212-reflections-2026>

